# Ders 4 Genişletilmiş Notu: Tümevarımla Yerine Koyma Yöntemi ve Hash Tabloları

## Ders 4 Genişletilmiş Notu: Tümevarımla Yerine Koyma Yöntemi ve Hash Tabloları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Geçen Dersin Tekrarı: Medyan Bulmanın Özyineleme Bağıntısı
- 2. Tümevarımla Yerine Koyma Yöntemi (Substitution Method)
- 3. Bağlı Listelerin Tekrarı
- 4. Sözlük, Doğrudan Adresleme ve Hashing
- 5. Zincirleme (Chaining) ile Çakışma Çözümü
- 6. Zincirlemeli Hashing'in Analizi
- 7. İyi Bir Hash Fonksiyonu: Bölme ve Çarpma Yöntemleri
- 8. Açık Adresleme (Open Addressing)
- 9. Açık Adresleme Teknikleri: Doğrusal, Karesel ve Çift Hashing
- Örnek Sorular


### 1. Geçen Dersin Tekrarı: Medyan Bulmanın Özyineleme Bağıntısı

Geçen hafta master teoremiyle analizler yapılmış, böl ve fethet (divide and conquer) algoritmalarından dışbükey örtü (convex hull) ve medyan bulma incelenmişti. Dışbükey örtünün karmaşıklığı master teoremiyle hesaplanabilmişti; medyan bulmada ise durum farklıydı ve başka bir yöntem gerekiyordu. Hoca bu derse o yöntemle başladı, ama önce analizde soru işareti kalmasın diye geçen haftaki algoritmayı kısaca hatırlattı.

Medyan, diziyi sıralamadan böl ve fethet ile bulunuyordu. Sözde kodun temel işi bir pivot seçmek ve pivottan küçük ve büyük elemanlarla iki grup oluşturmaktı. Bu işlem, aranan sıradaki eleman bulunana kadar özyinelemeli olarak tekrarlanıyordu. Pivot seçimi karmaşıklığı doğrudan etkilediği için pivotun akıllıca seçilmesi gerekiyordu; bunun için medyanların medyanı yöntemi kullanıldı. Pivotu seçmek için de yeniden medyan bulmak gerektiğinden bir durma noktası tanımlanmalıydı: eleman sayısı belirli bir sabitin altına inince medyan sabit zamanda (örneğin sıralayarak) bulunur ve algoritmaya buna uygun eklemeler yapılır.

Analizde, iki parçadan birinde olabilecek en fazla eleman sayısından yola çıkılmıştı. Her biri 5 elemanlı sütunlar vardı; medyanların medyanı $x$ ise $x$'ten büyük ve küçük eleman sayıları bir eşitsizlikle hesaplanmıştı. Algoritma pivotun yalnız bir tarafıyla ilgilenir, öbür parçayı atar (discard). Recurrence yazılırken kalan parçanın en büyük olabileceği durum düşünülür ve bu $7n/10+6$'yı verir. Dizi 5'erli bölündüğü için $n/5$ sütun vardır; medyanların medyanını bulmak için özyineleme $T(n/5)$ getirir. Bir de doğrusal iş vardır: her sütunun 5 elemanı sıralanmalıdır. Her sıralama $O(1)$'dir (5'e indirmenin avantajı budur), ama $n/5$ sütun olduğu için toplamı $n$ mertebesindedir.

Bu bağıntıya master teoremi uygulanamaz, çünkü master teoremi bölmede alt problemlerin eşit olmasını şart koşar. Özyineleme ağacıyla (recursion tree) çözmek de çok zorlaşır. Bu durumda tümevarım (mathematical induction) kullanılır.

## 2. Tümevarımla Yerine Koyma Yöntemi (Substitution Method)

### 2.1 Yöntemin Adımları

Derste işlenen konunun adı "substitution method that uses mathematical induction"dır. Tümevarımla ispatta önce bulunmak istenen karmaşıklık hakkında bir tahminde bulunulur. Yöntemin zor tarafı budur: önce doğru bir tahmin yapmak, sonra bazı varsayımlarla bu tahminin doğruluğunu ispatlamak gerekir.

- **1. adım:** Çözümün biçimi tahmin edilir. Bu önemli bir adımdır; işlemlerde tahminin doğru olduğu kabul edilir.
- **2. adım:** Tahminin $n$'den küçük değerler (1, 2, 3, ..., $n-1$ ya da bağıntı neye bağlıysa o değerler) için doğru olduğu varsayılır ve $n$ için doğru olduğu kanıtlanır.

Bağıntının değişkeni her zaman eleman sayısı olmak zorunda değildir; bağıntı neye bağlıysa o değişken üzerinden çalışılır.

### 2.2 Örnek: Merge Sort'un Bağıntısı

Hoca yöntemi tanıdık bir bağıntıyla gösterdi: merge sort'un özyineleme bağıntısı. Bu bağıntı master teoremiyle çözmeye çok uygundur, ama basit olduğu için tümevarım anlatımlarında genelde bunun üzerinden gidilir (ekrandan, 00:30:00'daki slayt listesi, 1. slayt):

$$T(n) = 2T(\lfloor n/2 \rfloor) + n \quad (n > 1), \qquad T(1) = 1$$

İkiye bölerek giden bir işlem olduğu için tahmin $T(n) = O(n \log n)$'dir. İlk haftalarda verilen Big O tanımına göre bu, bir $c$ katsayısıyla $cn \log n$'nin $T(n)$'den büyük olması demektir. İspatlanacak olan:

$$T(n) \leq cn \log n$$

Varsayım: $n$'den küçük her $m$ için $T(m) \leq cm \log m$.

Bağıntıda $m = n/2$ alınırsa varsayımdan $T(n/2) \leq c\frac{n}{2} \log \frac{n}{2}$ elde edilir. Bu, bağıntıdaki $T(n/2)$'nin yerine konur. $T(n/2)$ ondan büyük bir ifadeyle değiştirildiği için eşittir yerine "küçük eşit" yazılır:

$$T(n) \leq 2\left(c\frac{n}{2} \log \frac{n}{2}\right) + n = cn \log \frac{n}{2} + n$$

Logaritma kuralı $\log \frac{a}{b} = \log a - \log b$ ve $\log_2 2 = 1$ kullanılarak:

$$T(n) \leq cn(\log n - 1) + n = cn \log n - cn + n = cn \log n - n(c-1)$$

Hoca burada eksi işaretli bir terim çıktığını, bunun ispat anlamına geldiğini söyledi: $cn \log n$'den bir şey çıkarılıyor, yani sonuç $cn \log n$'yi aşamaz; $n$ ve $c$ pozitif olduğu için ifade negatif bir şeye doğru gitmez. Sonuç olarak $T(n) \leq cn \log n$.

> **Ders sonrası not:** Son adımın geçerli olması için $n(c-1) \geq 0$, yani $c \geq 1$ gerekir; yalnız $c$'nin pozitif olması yetmez. Aşağıda seçilen $c = 2$ bu koşulu sağlar.

### 2.3 Başlangıç Değerleri: $c$ ve $n_0$

Asimptotik notasyon tanımı gereği eşitsizliğin $n \geq n_0$ değerleri için çalıştığı gösterilmelidir. Peki $n_0$ ne olmalı? (ekrandan, 00:26:40)

- $n_0 = 0$ verilemez, pozitif olmalıdır.
- $n_0 = 1$: $T(1) = 1$ ama $c \cdot 1 \cdot \log 1 = 0$; $T(1) \leq 0$ sağlanmaz.
- $n \geq 2$ denenir: $T(2) = 2T(1) + 2 = 4$, $T(3) = 2T(1) + 3 = 5$. Bu değerlerde sorun yoktur.

Sonra bu kısıtları sağlayan $c$ aranır:

- $c = 1$: $n = 2$ için $T(2) \leq 1 \cdot 2 \log 2 = 2$, yani $4 \leq 2$. Sağlanmaz.
- $c = 2$: $T(2) \leq 2 \cdot 2 \log 2 = 4$, yani $4 \leq 4$. Sağlanır. $n = 3$ için de tutar.

Böylece $c = 2$ ve $n_0 = 2$ için eşitsizliğin tutarlı olduğu, dolayısıyla tahminin doğru olduğu görülür. Merge sort'un karmaşıklığının $n \log n$ olduğu tümevarımla ispatlanmış olur.

> **Ders sonrası not:** $c = 2$ ile $T(n) \leq 2n \log_2 n$ eşitsizliği, $T(1) = 1$ ve taban fonksiyonlu bağıntıyla $2 \leq n < 5000$ aralığındaki bütün $n$ için python ile doğrulandı ($n = 3$ için $5 \leq 6 \log_2 3 \approx 9{,}5$).

### 2.4 Soru-Cevap: Neden $n + 1$ Değil de $n/2$?

**Soru:** İkinci adımda $n$ için bir fazlasını kanıtlamaya çalışmıyor muyduk? Burada ikiye bölerek gittik; orası tam anlaşılmadı.

**Cevap:** Bu bağıntı $n - 1$, $n - 2$ diye değil, ikiye bölünerek ilerliyor. Bu yüzden bir sonraki adımda $m = n/2$ için varsayım kullanıldı. Elde edilen eşitsizlik, $n$'nin diğer değerleri için de geçerli olup olmadığını zaten gösteriyor; tahmin yanlış olsaydı eşitsizlik bunu ortaya çıkarırdı (bir sonraki örnek bunu gösteriyor). Başlangıç değerlerinde de eşitsizliğin hangi değerden itibaren tuttuğu denenerek geçerli $c$ ve $n_0$ bulunuyor.

### 2.5 Yanlış Tahmin Yapılırsa

Aynı bağıntı için tahmin $T(n) = O(n)$, yani $T(n) \leq cn$ olsaydı ne olurdu? (ekrandan, 00:31:30) Varsayım $T(m) \leq cm$ olur. $m = n/2$ için yerine konunca:

$$T(n) = 2T(n/2) + n \leq 2\frac{cn}{2} + n = cn + n$$

Elde edilen $T(n) \leq cn + n$'dir. Bunun $T(n) \leq cn$'yi vermesi ancak $n$ negatif olsaydı mümkün olurdu. $n$ pozitif olduğu için bu imkânsızdır; demek ki tahmin yanlıştır. Hoca, tümevarımın yanlış tahmini bu şekilde ortaya çıkardığını vurguladı.

### 2.6 Medyan Bulmanın Analizi

Hoca, tümevarımın master teoremi kadar pratik olmadığını, hataya açık bir iş olduğunu, ama master teoremi ya da özyineleme ağacıyla çözülemeyen durumlarda işe yaradığını söyledi. Medyan bulmanın bağıntısı (ekrandan, 00:40:00):

$$T(n) = \begin{cases} O(1) & n < 120 \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & n \geq 120 \end{cases}$$

Hoca sabit eşik için 120, 140, 150 gibi bir değer seçilebileceğini, bu eşiğin altında problemin sabit zamanda çözüleceğini hatırlattı. (Hoca ilk yazarken eşitsizlik yönünü ters yazdığını fark edip düzeltti.)

Hedef $T(n) = O(n)$'yi ispatlamaktır. Tahmin $T(n) \leq cn$. Varsayımlar:

$$T(n/5) \leq c\frac{n}{5}, \qquad T(7n/10 + 6) \leq c\left(\frac{7n}{10} + 6\right)$$

Hoca, şimdiye kadar hep tek parçalı bağıntılarla çalışıldığını, burada iki parçalı bir bağıntının birleşiminin analiz edildiğini belirtti. Sonucun $O(n)$ olabilmesi için iki parçanın da $O(n)$'e uyması, yani iki varsayımın da tutarlı olması gerekir. Sondaki doğrusal iş $an$ diye bir sabitle yazılır; bu kısım özyinelemeli değil sabit bir işlem olduğu için ayrı tutulur:

$$T(n) \leq c\frac{n}{5} + c\left(\frac{7n}{10} + 6\right) + an = \frac{cn}{5} + \frac{7cn}{10} + 6c + an = \frac{9cn}{10} + 6c + an$$

Merge sort örneğinde, varsayılan sınırdan sonra bir çıkarma işlemi gelmesi ispata zemin hazırlamıştı. Burada artı işaretli terimler var, bu yüzden daha dikkatli düşünmek gerekiyor. İspatlanmak istenen ifade $cn$ olduğu için ifade $cn$ ile başlayacak biçimde yazılır: $9cn/10 = cn - cn/10$.

$$T(n) \leq cn + \left(-\frac{cn}{10} + 6c + an\right) = cn - \left(\frac{cn}{10} - 6c - an\right)$$

Merge sort'takine benzer bir yapı çıktı. $T(n) \leq cn$ olmasının tek şartı parantez içindeki ifadenin negatif olmamasıdır (ekrandan, 00:42:40):

$$\frac{cn}{10} - 6c - an \geq 0 \;\Rightarrow\; \frac{cn}{10} \geq 6c + an \;\Rightarrow\; cn \geq 60c + 10an \;\Rightarrow\; c \geq \frac{60c}{n} + 10a$$

Bu noktada $n$ için bir sınır, $c$ için de $a$ cinsinden bir sınır belirlemek gerekir; bu biraz deneyerek yapılabilir. Hoca eşitlik durumunu düşünmenin katsayıları bulmayı kolaylaştıracağını söyledi ve $n \geq 120$, $c \geq 20a$ seçti. $c = 20a$, $n = 120$ yerine konunca:

$$20a \geq \frac{60 \cdot 20a}{120} + 10a = 10a + 10a = 20a$$

Eşitlik sağlanır. Böylece $c \geq 20a$ ve $n \geq 120$ için tümevarımın tuttuğu gösterilir. Hoca bunun bir anlamını da vurguladı: algoritmaya $O(1)$'de çözülecek bir durma eşiği konması gerekiyordu; o eşiğin ne olması gerektiği de bu hesapta görülmüş olur.

> **Ders sonrası not:** $c \geq 60c/n + 10a$ eşitsizliğinde $c$ iki tarafta da var; $c$ yalnız bırakılırsa $c \geq \frac{10an}{n-60}$ ($n > 60$) çıkar. Bu alt sınır $n$ büyüdükçe küçülür ($n = 120$ için $20a$, $n = 140$ için $17{,}5a$), bu yüzden $n \geq 120$ için $c = 20a$ yeterlidir. $c = 20a$ ile $cn/10 - 6c - an \geq 0$ koşulu $n = 120, 121, 140, 500, 10^6$ için python ile kontrol edildi.
>
> **Ek bilgi (derste anlatılmadı):** İspatta $T(\lceil n/5 \rceil)$ yerine $c\,n/5$ yazıldı, yani tavan fonksiyonu göz ardı edildi. Titiz bir ispatta $\lceil n/5 \rceil \leq n/5 + 1$ kullanılır ve fazladan bir $c$ terimi gelir; bu yüzden ders kitaplarında (Cormen ve arkadaşları) eşik büyük olasılıkla 140 olarak, aynı $c \geq 20a$ koşuluyla verilir. Geçen dersin slaytında da eşik 140 idi.

## 3. Bağlı Listelerin Tekrarı

İkinci bölümde hash fonksiyonlarına geçilecekti; hash tablolarının bir çeşidi bağlı listeleri kullandığı ve ispatlarda onlara bakılacağı için hoca önce bağlı listeleri hızlıca hatırlattı.

### 3.1 Tek Yönlü Bağlı Liste

Bağlı listeler bir açıdan dizilere benzer: aynı tipteki verileri tutar. Ama elemanlar düğümlerdir (node) ve birbirlerini işaret ederler (point). Bu, silme ve ekleme işlemlerini kolaylaştırır; listenin boyutunun dinamik olarak artıp azalmasını da kontrol etmeyi kolaylaştırır. İlk düğüm (first node, head) önemlidir. Her düğüm, bir sonraki düğümün adresini kendi next alanında tutar. Listenin sonunu yakalamak için son düğümün NULL'ı gösterdiği kabul edilir ve böyle gerçeklenir.

Slayttaki örnekte dört düğüm vardı; her düğüm next alanıyla bir sonrakini gösteriyordu. Bu tek yönlü bağlı listedir (singly linked list): her düğüm yalnız bir sonrakini gösterir, geriye gitmek mümkün değildir, hep ilerlemek gerekir.

- **Ekleme:** Önce yeni düğüm oluşturulur. Sonra öncesindeki ve sonrasındaki elemanların adresleri üzerinde işlem yapılır: yeni düğümün adresi önceki elemanın next alanına yazılır; yeni düğümün next'i ise önceden A'nın gösterdiği düğümü gösterir.
- **Silme:** A ile B arasındaki X düğümü silinecekse, A'nın gösterdiği adresin X yerine B olması sağlanır. Böylece X devreden çıkar ve liste A'dan doğrudan B'ye geçer.

### 3.2 C Kodları

Hoca işaretçileri hatırlatmak için C kodları gösterdi. Önce kendi tipinde bir işaretçi tutan bir düğüm yapısı tanımlanır; buradaki basit düğüm tek bir değer ve bir işaretçi tutar (taşınacak bilginin büyüklüğüne göre yapı genişler).

- **Sona ekleme:** Fonksiyon eklenecek değeri ve head'i alır. İşaretçi parametre olarak aktarıldığında main'deki head'in değişmemesi için `current` adlı yedek bir işaretçiyle ilerlenir. Her adımda sona ulaşılıp ulaşılmadığı kontrol edilir. Sona gelince yeni düğüme yer açılır, açılan yerin adresi son düğümün next'ine atanır; yeni düğümün değer alanına değer, next'ine NULL yazılır. Böylece listenin sonu değişmiş olur.
- **Başa ekleme:** Yeni düğüm oluşturulur ve yer açılır. Yeni düğümün next'i önceki head'i gösterir ve değer yazılır. Head değiştiği ve diğer fonksiyonlar listeye head üzerinden ulaşacağı için yeni head döndürülür (return). Hoca bunun bir tasarım olduğunu, başka yolların da olabileceğini söyledi.
- **Listeleme:** Head alınır, gezmek için bir değişken tanımlanır, sona gelinip gelinmediğine bakılarak her düğümün değeri ekrana yazılır ve sonraki düğüme geçilir. Arama (search) için de benzer bir yapı kullanılır.
- **Silme:** Silme fonksiyonuna düğümün adresi değil değeri verilir, bu yüzden düğüm aranmalıdır. Listenin sonuna gelene kadar ya da aranan değer bulunana kadar dönülür. Önceki düğüm de tutulmalıdır, çünkü önceki ve sonraki düğüm birbirine bağlanacaktır.

Silme kodu slaytta şöyleydi (ekrandan, 01:00:00):

```
void deleteNode(struct node *head, int val)
{
    struct node* current = head, *before=NULL;

    while ((current->val != val)&&(current->next != NULL))
    {
        before=current;
        current = current->next;
    }

    if (current->val != val)
        printf("silinmek istenen eleman listede yok\n");
    else
    {
        before->next=current->next;
        free (current);
    }
}
```

Hoca, silinen düğümün bellekten silinebileceğini (`free`) ya da kalması isteniyorsa farklı bir işaret konabileceğini söyledi. Bu parça kod bir yapı tanımı ve `main` içermediği için derlenmedi.

> **Ek bilgi (derste anlatılmadı):** Bu fonksiyonda silinecek değer ilk düğümdeyse döngüye hiç girilmez ve `before` NULL kalır; `before->next` erişimi hataya yol açar. Baştaki düğümü silmek için head'in güncellenip döndürülmesi (başa eklemedeki gibi) ayrıca ele alınmalıdır.

### 3.3 Çift Yönlü Bağlı Liste

Hoca çift yönlü bağlı listeyi (doubly linked list) gerçeklemeye vakit ayırmadı. Burada her düğüm hem öncekine hem sonrakine referans verir; bu hareket kolaylığı sağlar ve bazı işlemleri daha rahat ve hızlı yapmayı mümkün kılar.

- **Silme:** Tek yönlü listede önceki ve sonraki düğüm bağlantısını kurmak tek bir işlemdi. Çift yönlüde silinen düğümün öncesinin next'i silinen düğümün next'ini göstermeli, bunun tersi de yapılmalıdır: iki bağlantı birden güncellenir. Sonra eski düğüm silinir.
- **Ekleme:** Yeni düğüm oluşturulur. Yeni düğümün önceki düğümü gösteren adresine mevcut düğüm, sonrakini gösteren adresine mevcut düğümün gösterdiği düğüm konur. Sonra komşu düğümlerin ilgili bağlantıları da güncellenir.

# 4. Sözlük, Doğrudan Adresleme ve Hashing

## 4.1 Sözlük ve Anahtar

Sözlük (dictionary), elemanları üzerinde search, insert ve delete işlemlerinin tanımlı olduğu bir kümedir. Elemanlar sayılar, karakterler, linkler vb. olabilir. Pek çok dilde (örneğin Java'da) bir veri tipi olarak bulunur. Sözlükler üzerinde işlem yapmak için hash fonksiyonları oldukça kullanışlıdır.

Tipik bir kayıtta (record) bir varlığın belirli özellikleri alanlarda (field) tutulur; örneğin bir öğrenci için öğrenci numarası. Alanlardan en az biri varlığı ayırt etmek için kullanılır: anahtar (key). $n$ kayıttan oluşan bir sözlük gerçeklenecekse, anahtarları $K_1, K_2, K_3, ...$ olan kayıtlara rahat ulaşmak için bunlar bir hash tablosunda tutulabilir.

Hashing, anahtarların bir boyutlu bir diziye dağıtılması fikrine dayanır. Dağılım, her anahtar için önceden tanımlanmış bir hash fonksiyonunun hesaplanmasıyla yapılır. Hash fonksiyonu anahtara $0$ ile $m-1$ aralığında bir tam sayı atar ($m$ aynı zamanda tablonun boyutudur); bu sayıya hash adresi denir.

Insert, search, delete gibi sözlük işlemlerini destekleyen dinamik kümelere ihtiyaç duyan pek çok uygulama vardır. Örneğin derleyicilerdeki sembol tabloları çoğu zaman hash tablolarıyla gerçeklenir, çünkü derleyicinin bir sembolle ilgili veriye hızla ulaşması gerekir. Kriptolojide de hash tablolarının çok kullanımı vardır. Hash tablosunda arama en kötü durumda $O(n)$'e çıkabilir, ama standart durumlarda bu işlemler sabit zamanda, $O(1)$'de yapılabilir. Hash tablosu tipik olarak gerçekten saklanan anahtar sayısıyla orantılı boyutta bir dizi kullanır. Saklanan anahtar sayısı olası anahtarların toplam sayısına göre küçük olduğundan hash tabloları, diziyi doğrudan adreslemeye güzel bir alternatiftir.

## 4.2 Doğrudan Adresleme (Direct Addressing)

Doğrudan adresleme hashing'in ilk versiyonu gibidir. Bir anahtar evreni vardır ve adresleme doğrudan anahtarlar kullanılarak yapılır. Evren çok büyük olmayan sayılardan oluşuyorsa ve iki elemanın aynı anahtara sahip olmadığı düşünülüyorsa doğrudan adresleme yapılabilir. Gerçeklemesi kolaydır ve sabit zamanda erişim sağlar.

Slayttaki şekilde (ekrandan, 01:10:00) $U$ anahtar evreni (0-9), $K$ gerçekten kullanılan anahtarlar (2, 3, 5, 8) ve $T$ doğrudan adres tablosu vardır. Tablonun 2, 3, 5 ve 8 numaralı slotları anahtarı ve ek veriyi (satellite data) tutan yapıları gösterir, diğerleri boştur. 2 anahtarı tablonun 2. elemanına gider ve oradaki bilgiye ulaşılır; 3 kullanılırsa tek adımda 3 ile ilgili bilgiye gidilir.

Search, insert ve delete işlemlerinin hepsi $O(1)$'dir: arama `return T[k]` ile doğrudan adrese erişir; ekleme $x$'in anahtarı üzerinden yapılır (örneğin anahtar 5 ise $x$ kaydının adresi $T[5]$'e yazılır); silmede o slota NULL konur.

**Soru (hocanın sorusu):** $m$ uzunluğundaki doğrudan tabloda tanımlanmış bir $S$ dinamik kümesinde en büyük elemanı bulmak gibi bir işlemin en kötü durumu nedir? **Cevap:** Bütün tabloyu taramak gerekir; $O(m)$.

Doğrudan adreslemenin sakıncası tablo boyutunun uygun olmamasıdır. Slayttaki ifadeyle: evren $U$ büyükse $|U|$ boyutlu bir $T$ tablosunu saklamak tipik bir bilgisayarın belleğiyle pratik olmayabilir, hatta imkânsız olabilir. Ayrıca gerçekten saklanan anahtar kümesi $K$, $U$'ya göre o kadar küçük olabilir ki $T$ için ayrılan alanın çoğu boşa gider. Anahtarlar birbirine yakın değerlerdeyse ve belirli bir aralığın altında oldukları biliniyorsa doğrudan adresleme iyidir, gerçeklemesi de kolaydır. Bu sorunlar yüzünden anahtarı bir hash fonksiyonuyla tabloya yerleştirme fikri ortaya çıkmıştır.

## 4.3 Hashing ve Çakışma (Collision)

Doğrudan adreslemede anahtar doğrudan $T$'nin ilgili slotuna konuyordu. Hashing'de ise eleman, bir fonksiyonun sonucunun gösterdiği yere konur. Böylece dizinin boyutu küçülür; bütün anahtarların bulunduğu evrenden daha küçük bir tabloyla iş görülebilir.

Ama bir hash fonksiyonu iki ayrı anahtarı aynı yere eşleyebilir; buna çakışma (collision) denir. Çakışmaya bulunan çözümler konunun temelini oluşturur. Bir hash fonksiyonunun iyiliği, anahtarları rastgele bir şekilde dağıtabilmesiyle orantılıdır: ne kadar rastgeleye yakın dağıtırsa o kadar az çakışma çıkar. Rastgele görünümlü (random looking) hash fonksiyonu bulmak için farklı yöntemler vardır; en basit hâllerinde asal sayılardan yararlanılır. Yine de çakışmalar ortaya çıkar ve çözülmesi gerekir. Güzel bir yol bağlı listeleri kullanmaktır.

## 5. Zincirleme (Chaining) ile Çakışma Çözümü

Zincirlemede (chaining) tablodaki ilgili gözlere bağlı listeler bağlanır: çakışan elemanlar bir bağlı listede tutulur ve $T$'nin ilgili elemanı listenin başını gösterir. Slayttaki örnekte $k_1$ ve $k_4$ aynı slota düşmüş, $k_5$, $k_7$ ve $k_2$ aynı listede, $k_8$ ve $k_6$ bir başka listede yer almış, $k_3$ tek başına kalmıştır.

**Soru (hocanın sorusu):** Listede neden tek yönlü değil çift yönlü bağlı liste kullanılıyor? **Cevap:** Çift yönlü liste silmeyi çok kolaylaştırır ve hızlandırır.

İşlemlerin karmaşıklıkları:

- **Ekleme:** Karmaşıklığı düşürmenin yolu yeni düğümü listenin sonuna değil başına eklemektir. Çakışma varsa yeni eleman, o slotun gösterdiği ilk adrese, yani listenin head'ine konur; ekleme sabit zamanda, $O(1)$'de yapılır.
- **Arama, en kötü durum:** İyi bir hash fonksiyonu kullanılmadıysa ya da elemanlar kötü niyetli olarak, bilerek aynı yeri üretecek şekilde geldiyse, bütün elemanlar tek bir listede bulunur. O liste taranacağı için $O(n)$.
- **Silme:** Çift yönlü liste kullanıldığı için silinecek elemanın adresi verildiğinde öncesindeki ve sonrasındaki elemanların previous ve next alanları birbirine bağlanır. Adres bilgisi elde olduğundan iş $O(1)$'dir: düğüm verildiğinde onun bağlı olduğu önceki ve sonraki elemanlar da elde vardır.

**Soru (hocanın sorusu):** Çift yönlü değil tek yönlü liste kullanılsaydı silmenin karmaşıklığı ne olurdu? **Cevap:** $O(n)$. Bütün elemanların aynı hash değerine sahip olduğu senaryoda, düğümün kendisi verilmiş olsa bile önceki elemanı bilmek gerekir ki sonraki elemanla bağlansın. Tek yönlü düğüm yalnız next'i tutacak biçimde tasarlandığından önceki eleman ancak liste baştan taranarak bulunur.

## 6. Zincirlemeli Hashing'in Analizi

### 6.1 Yük Faktörü (Load Factor)

Çakışmaları zincirlemeyle çözen hash tablolarının analizinde kullanılan değişkenler (ekrandan, 01:30:00'daki 1. slayt):

- $T$: hash tablosu.
- $m$: tablodaki slot sayısı (number of slots).
- $n$: tabloda saklanması planlanan eleman sayısı (number of stored elements).

Analizi doğrudan etkileyen şey $n$ ile $m$'nin oranıdır. $T$ için yük faktörü (load factor):

$$\alpha = \frac{n}{m}$$

$n$ toplam eleman, $m$ slot sayısı olduğu için yük faktörü bir zincirdeki ortalama eleman sayısını (average number of elements stored in a chain) verir. Bu dağılım her zaman düzgün olmayabilir. $\alpha$ 1'den küçük, 1'e eşit ya da 1'den büyük olabilir.

En kötü durumda bütün anahtarlar aynı slota düşerse arama $O(n)$'dir, çünkü oraya gidip bütün elemanları gezmek gerekir. En iyi durumda iş $O(1)$'dir. İlgilenilen şey ortalama durum (average case) performansıdır ve bu, yük faktörüne bağlıdır.

Ortalama durum hesaplanırken her elemanın $m$ slottan herhangi birine yerleşme olasılığının eşit olduğu varsayılır. Buna basit düzgün hashing (simple uniform hashing) denir.

### 6.2 Başarısız Arama

**Teorem:** Çakışmaların zincirlemeyle çözüldüğü bir hash tablosunda, basit düzgün hashing varsayımı altında başarısız (unsuccessful) bir arama ortalama $\Theta(1+\alpha)$ sürer.

Hoca işleyişi gözde canlandırmak için bir tablo çizdi (ekrandan, 01:30:00'daki 2. slayt). Çakışma olduğunda eklenecek eleman, ilgili slottaki bağlı listenin en başına konuyordu. Önce $x_1$, sonra $x_2$, sonra $x_3$ aynı slota yerleştirilmiş olsun. $x_2$ gelince çakışma olmuş ve $x_1$'in önüne, $x_3$ gelince yine çakışma olmuş ve en başa konmuştur. Liste $x_3 \to x_2 \to x_1$ biçimindedir (çift yönlü).

$x_1$ aranırsa, $x_1$'den sonra yerleştirilen elemanlar ve $x_1$'in kendisi taranır: $2+1=3$ işlem. Bu sayı zincirdeki eleman sayısına denk gelir. Başarısız bir arama ise listenin sonuna kadar gitmeyi gerektirir. Yük faktörü bir zincirdeki ortalama eleman sayısını verdiğine göre, başarısız bir arama için üzerinden geçilmesi gereken eleman sayısı ortalama $\alpha$'dır; hash değerinin hesaplanmasıyla birlikte $\Theta(1+\alpha)$.

### 6.3 Başarılı Arama

**Teorem:** Zincirlemeli bir hash tablosunda başarılı (successful) bir arama, düzgün hashing varsayımıyla ortalama $\Theta(1+\alpha)$ sürer (ekrandan, 01:33:40).

Başarısız aramada da $\Theta(1+\alpha)$ idi; başarılıda da aynısı mı olacak? İspat:

$x_i$ tabloya eklenen $i$. eleman olsun. $X_{ij}$ rastgele değişkeni şöyle tanımlanır:

$$X_{ij} = \begin{cases} 1 & x_i \text{ ve } x_j \text{ aynı slota düşüyorsa} \\ 0 & \text{aksi hâlde} \end{cases}$$

$x_i$ eklendikten sonra eklenenlerden $x_i$ ile aynı hash değerini verenlerin sayısı:

$$\sum_{j=i+1}^{n} X_{ij}$$

Eşitse toplama 1 katar, değilse katmaz. Yeni elemanlar listenin başına eklendiği için bunlar $x_i$'nin önündedir ve $x_i$ aranırken onlar da incelenir. Bu durumda başarılı aramadaki ortalama inceleme (examination) sayısı:

$$\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n} X_{ij}\right)$$

Buradaki 1, $x_i$'nin kendisi için yapılan incelemedir; iç toplam, ondan önce gelen elemanların sayısıdır. Parantez her $i$ için yapılacak inceleme sayısını verir; $\frac{1}{n}$ ile çarpmak ortalamayı verir.

Burada beklenen değer (expected value) kavramı devreye girer; ifade etmeyi kolaylaştırdığı için kaynaklar bunu kullanır. $X_{ij}$'nin beklenen değeri, $x_i$ ile $x_j$'nin aynı slotta bulunma olasılığıdır. $m$ slot sayısı olduğu için:

$$E[X_{ij}] = \frac{1}{m}$$

Hesap (ekrandan, 01:40:00):

$$\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\frac{1}{m}\right) = \frac{1}{n}\sum_{i=1}^{n}\left(1+\frac{n-i-1+1}{m}\right) = \frac{1}{n}\sum_{i=1}^{n}\left(1+\frac{n-i}{m}\right)$$

İç toplamda $j$, $i+1$'den $n$'e kadar gider; terim sayısı $n-(i+1)+1 = n-i$'dir. Toplam sembolü içeri taşınır; 1'lerin toplamı $n$ verir:

$$= \frac{1}{n}\left(n + \sum_{i=1}^{n} \frac{n-i}{m}\right) = 1 + \frac{1}{nm}\sum_{i=1}^{n}(n-i)$$

Toplam formülüyle $\sum_{i=1}^{n}(n-i) = n^2 - \frac{n(n+1)}{2}$:

$$= 1 + \frac{1}{nm}\left(n^2 - \frac{n(n+1)}{2}\right) = 1 + \frac{n-1}{2m}$$

Yük faktörü $\alpha = n/m$ cinsinden:

$$1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

$\frac{\alpha}{2} - \frac{\alpha}{2n} = \frac{1}{2}\left(\alpha - \frac{\alpha}{n}\right)$, yani $\alpha$'nın sabit bir katsayıyla çarpımıdır. Sabit katsayılar notasyonda atıldığı için sonuç $\Theta(1+\alpha)$'dır. Başarılı aramalar da ortalama durumda $1+\alpha$ işlemle yapılır.

> **Ders sonrası not:** Slaytın ikinci satırında payda $n$ gibi yazılmış görünüyor (ekrandan, 01:40:00); doğrusu $m$'dir, bir önceki satırdaki $1/m$'den gelir. Formülün son hâli $1 + (n-1)/(2m)$, $n = 50$, $m = 7$ için doğrudan toplamla python'da karşılaştırıldı; ikisi de $4{,}5$ verdi.

## 7. İyi Bir Hash Fonksiyonu: Bölme ve Çarpma Yöntemleri

### 7.1 İyi Hash Fonksiyonunun Özellikleri

Hash fonksiyonunun iyi kabul edilmesi için anahtarların dağılımı iyi, yani rastgeleye yakın olmalıdır (ekrandan, 01:50:00). Bir hash tablosu için fonksiyon tasarlanırken "pt", "pts" gibi birbirine çok yakın stringlerde sorun yaşanmamasını gözeten bir fonksiyon seçilmelidir. Bu bazen probleme bakılarak karar verilecek bir şeydir; bazen de çok kolaydır: anahtarlar 0 ile 1 arasındaki sayılarsa onları bir $m$ değeriyle çarpmak düzgün dağılımlı bir hash fonksiyonu verir. Hash fonksiyonu oluşturmak için birkaç yöntem vardır: bölme yöntemi (division method) ve çarpma yöntemi (multiplication method).

### 7.2 Stringlerin Sayıya Çevrilmesi

Çoğu hash fonksiyonu doğal sayılar üzerinde çalışır. Bu yüzden stringleri doğal sayı olarak yorumlayacak bir düzenleme gerekir: radix gösterimi, yani taban sistemi. Örneğin "pt" stringi, harflerin ASCII kodları ve taban (radix) 128 kullanılarak sayıya çevrilebilir. p'nin kodu 112, t'nin kodu 116'dır. İki karakter olduğu için daha az önemli basamak $128^0$, diğeri $128^1$ ile çarpılır (daha fazla karakter olsaydı $128^2$, $128^3$ diye gidecekti):

$$\text{"pt"} = 112 \cdot 128^1 + 116 \cdot 128^0 = 14336 + 116 = 14452$$

(Sonuç slayttaki hesapla aynıdır ve python ile doğrulandı; p ve t'nin ASCII kodları 112 ve 116'dır.)

### 7.3 Bölme Yöntemi (Division Method)

Bölme yönteminde hash fonksiyonu bir bölme işlemiyle üretilir (ekrandan, 01:52:55):

$$h(k) = k \bmod m$$

$m$'nin seçimi fonksiyonun başarısı için kritiktir. 2'nin tam kuvvetine yakın olmayan bir asal sayı seçmek mantıklıdır. Kaç elemanla çalışıldığı da önemlidir.

Örnek: $n = 2000$ eleman tutacak bir hash tablosu tasarlanacak. Başarısız bir aramada kaç elemanın incelenmesinin performansı fazla etkilemeyeceğine karar verilir; örneğin 3. O zaman $2000/3 \approx 667$'ye yakın ve 2'nin kuvvetlerinden uzak bir asal seçilir: $m = 701$. Asal sayı kullanmak her zaman düzgün dağılımı destekler.

> **Ders sonrası not:** Hoca bunu "üç defa başarısız arama yapmak performansı çok etkilemeyecek" diye ifade etti (01:52:23). Kastedilen, başarısız bir aramada ortalama 3 elemanın incelenmesini göze almaktır; yani zincir başına ortalama 3 eleman, $\alpha \approx 3$ ve $m \approx n/3$.

### 7.4 Çarpma Yöntemi (Multiplication Method)

Çarpma yönteminin adımları (ekrandan, 02:00:30'daki 6. slayt):

1. Anahtar $k$, $0 < A < 1$ aralığındaki bir $A$ sabitiyle çarpılır.
2. $kA$'nın ondalık kısmı alınır.
3. Bu değer $m$ ile çarpılır.
4. Sonuç aşağı yuvarlanır (floor).

Sembollerle:

$$h(k) = \lfloor m\,(kA \bmod 1) \rfloor$$

Burada "mod1", sayının virgülden sonraki kısmını almak demektir. Bu yöntemde $m$'nin nasıl seçildiği çok kritik değildir; $A$'nın seçimi önemlidir. Önerilen değer:

$$A = \frac{\sqrt{5}-1}{2} \approx 0{,}618033$$

Hoca öğrencilere bunun ne olduğunu sordu: altın oran (golden ratio). Bu öneriyi, Turing ödülü almış ve algoritma analizinin babası kabul edilen bir araştırmacıya bağladı ve onun çalışmalarına göz atmayı önerdi.

> **Ders sonrası not:** Altyazıda adı bozuk geçen bu araştırmacı büyük olasılıkla Donald Knuth'tur; altın oran önerisi ders kitaplarında ona atfedilir.

**Örnek** (ekrandan, 02:00:30): $m = 10000$, $k = 123456$.

$$h(123456) = \lfloor 10000 \cdot (123456 \cdot 0{,}618033 \bmod 1) \rfloor$$

- $123456 \cdot 0{,}618033\ldots \approx 76300{,}0041151$
- Ondalık kısım: $0{,}0041151$
- $10000 \cdot 0{,}0041151 = 41{,}151$
- Aşağı yuvarlama: $h(123456) = 41$

Hesap python ile $A = (\sqrt{5}-1)/2$ kullanılarak doğrulandı: $123456 \cdot A = 76300{,}00411\ldots$, sonuç 41.

Hoca, universal hashing gibi başka hash fonksiyonu türlerinden sonraki derslerde bahsedeceğini söyledi.

## 8. Açık Adresleme (Open Addressing)

### 8.1 Fikir

Çakışma olduğunda zincirleme yapılıyordu. Diğer bir alternatif açık adreslemedir (open addressing). Açık adreslemede bütün elemanlar tablonun kendisinde tutulur; çakışanlar ayrı bir bağlı listeye konmaz. Bir elemanı yerleştirmek için önce bir slot denenir; orada yer yoksa başka bir slot denenir. Bu yüzden slotların deneme sırası belirlenmelidir.

Deneme sırasını belirleyen şey genişletilmiş bir hash fonksiyonudur. Şimdiye kadarki hash fonksiyonları yalnız anahtarı alıyordu; burada deneme sırasını gösteren ikinci bir parametre eklenir: $h(k, i)$. Her çakışmada bir sonraki slota yönlendirilir. Bir anahtar için deneme dizisi $h(k, 0), h(k, 1), \ldots, h(k, m-1)$'dir (ekrandan, 02:07:20'deki 8. slayt).

### 8.2 Ekleme (HASH-INSERT)

Anahtar ve tablo alınır, baştan ($i = 0$) başlanır. Hash fonksiyonu $i$ ve $k$ için bir $j$ değeri döndürür; bu slotun boş olup olmadığına bakılır. Boşsa (dolu, occupied değilse) eleman oraya yerleştirilir ve fonksiyondan çıkılır. Doluysa sonraki denemeye geçilir, $i$ artırılır; $i = 1, 2, \ldots$ için bakılır. Sona ulaşıldıysa tablo doludur ve hata verilir. Slayttaki sözde kod (ekrandan, 02:07:20'deki 8. slayt küçük görünümü; hoca yazarken bir yerde $j$ yerine yanlış harf yazdığını söyleyip düzeltti):

```
HASH-INSERT(T, k)
i = 0
repeat
    j = h(k, i)
    if T[j] == NIL
        T[j] = k
        return j
    else i = i + 1
until i == m
error "hash table overflow"
```

### 8.3 Arama (HASH-SEARCH)

Arama için de tablo ve aranan anahtar alınır. Hash fonksiyonunun ikinci parametresi 0 ile, yani ilk denemeyle başlanır; bir $j$ üretilir ve aranan elemanın orada olup olmadığına bakılır. Bulunduysa sorun yoktur; bulunamadıysa bir sonraki denemeyle devam edilir. Boş bir slota gelindiğinde ya da bütün slotlar denendiğinde durulur (ekrandan, 02:07:20):

```
HASH-SEARCH(T, k)
i = 0
repeat
    j = h(k, i)
    if T[j] == k
        return j
    i = i + 1
until T[j] == NIL or i == m
```

Temel fikir slotları tek tek denemek ve hash fonksiyonuna verilen ikinci parametreyle bir sonraki slotu elde etmektir.

> **Ek bilgi (derste anlatılmadı):** Slayttaki sözde kodda döngüden sonraki satır görünmüyor; standart hâlinde döngü elemanı bulamadan biterse fonksiyon NIL döndürür (aranan anahtar tabloda yok demektir).

### 8.4 Silme Sorunu

**Soru (hocanın sorusu):** Açık adreslemede silme nasıl yapılır?

**Cevap:** Silme sıkıntılıdır. Silinen elemanın yerine NIL konursa arama erken sonlanabilir. Arama normalde boş bir slota gelene kadar devam eder. Bir slot silinip NIL yapılırsa, aynı hash fonksiyonu bağlamında daha sonra yerleştirilmiş elemanlar ileride olduğu hâlde arama NIL'i görünce durur. Çözüm, NIL yerine DELETED gibi bir işaret (flag) koymaktır. Insert fonksiyonu DELETED yazan slotu boş kabul edecek şekilde güncellenir. Ancak silinen eleman çok olursa arama süresi yük faktöründen çok silinen elemanlara bağlı hâle gelir; bu bir dezavantajdır. Bu yüzden silme işlemine çokça ihtiyaç varsa açık adresleme iyi bir çözüm değildir; çakışmaları zincirlemeyle çözmek daha mantıklıdır.

## 9. Açık Adresleme Teknikleri: Doğrusal, Karesel ve Çift Hashing

Açık adresleme için üç teknik vardır (ekrandan, 02:20:00'deki 1. slayt): doğrusal deneme (linear probing), karesel deneme (quadratic probing) ve çift hashing (double hashing). Açık adreslemede hash fonksiyonu iki parametre alıyordu; bu tekniklerin hepsinde başlangıçta tek parametreli bir yardımcı hash fonksiyonu $h'(k)$ tanımlanır ve gerçekleme onun üzerinden yürütülür.

### 9.1 Doğrusal Deneme (Linear Probing)

Formül (ekrandan, 02:17:55):

$$h(k,i) = (h'(k) + i) \bmod m, \qquad i = 0, 1, \ldots, m-1$$

Yardımcı fonksiyonun çıktısı $i$ ile toplanır ve $m$'ye göre modu alınır. Doğrusal deneme her seferinde aynı tablo üzerinde slotlar arasında sırayla gezer. Yardımcı fonksiyon $h'(k) = k \bmod m$, $m = 7$ olsun.

- $k = 9$: $h'(9) = 9 \bmod 7 = 2$. $i = 0$: $(2 + 0) \bmod 7 = 2$. Slot 2 boş, 9 oraya konur.
- $k = 16$: $h'(16) = 16 \bmod 7 = 2$. $i = 0$: $h(16, 0) = (2 + 0) \bmod 7 = 2$; slot 2 dolu. $i = 1$: $h(16, 1) = (2 + 1) \bmod 7 = 3$; 16 slot 3'e konur.
- Yardımcı fonksiyonu yine 2 veren bir anahtar, örneğin 23 gelirse ($23 \bmod 7 = 2$), 2 ve 3 dolu olduğundan slot 4'e yerleşir; bir sonraki de 5'e, bir sonraki 6'ya.

Doğrusal denemenin avantajı gerçeklemesinin kolay olmasıdır. Ama kümelenme (clustering) denen bir sorun çıkar: arka arkaya dolu slotlardan kümeler oluşur. Kümeler, elemanların kendi beklenen slotlarına değil daha alttaki slotlara yerleşmesine yol açar; bu da arama süresini çok uzatır. Kümelenmeyi azaltacak bir çözüm gerekir.

### 9.2 Karesel Deneme (Quadratic Probing)

Karesel denemenin amacı, çakışma durumunda doğrusal değil karesel bir fonksiyon kullanmaktır. Herhangi bir karesel fonksiyon olabilir; slayttakiler örnektir (ekrandan, 02:24:25):

$$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

$$h(k,i) = (h'(k) + i^2) \bmod m$$

İlki başarılı bir karesel deneme fonksiyonudur; ikincisi daha basit bir biçimidir. Bu yöntem doğrusal denemeden daha iyi çalışır, ama tablonun tamamının kullanılabilmesi için $c_1$, $c_2$ ve $m$'nin seçilebilecek değerleri kısıtlıdır; dikkat edilmezse deneme dizisi tablonun bir kısmına sıkışabilir. İki anahtarın başlangıç konumu aynıysa ürettikleri deneme dizisi de aynı olur; yani kümelenme sorunu hâlâ vardır, ama daha kısa kümeler oluşur.

**Örnek:** $c_1 = 2$, $c_2 = 1$, $m = 7$, $h'(k) = k \bmod 7$. Doğrusal denemedeki 9, 16, 23 anahtarları arka arkaya yerleşmişti; karesel denemede ne olur?

- $k = 9$: $h'(9) = 2$; slot 2 boş, 9 oraya konur.
- $k = 16$: $h'(16) = 2$; slot 2 dolu. $i = 1$: $h(16, 1) = 2 + 2 \cdot 1 + 1 \cdot 1^2 = 5$; 16 slot 5'e konur.
- $k = 23$: $h'(23) = 2$; $h(23, 0) = 2$ dolu. $h(23, 1) = 2 + 2 \cdot 1 + 1 \cdot 1 = 5$ dolu. $h(23, 2) = 2 + 2 \cdot 2 + 1 \cdot 2^2 = 10$.

Hoca 23'ü tabloda bir yere yerleştirdi ve böylece kümelenmeyle daha az karşılaşıldığını, daha iyi bir dağılım sağlandığını söyledi; ama $c_1$, $c_2$ seçimi tablo dışına kaçmayacak biçimde olmalıdır.

> **Ders sonrası not:** Hoca $h(23, 2)$ için 10 buldu ve mod $m$ işlemini uygulamadı. $10 \bmod 7 = 3$'tür; slot 3 boş olduğu için 23 slot 3'e yerleşir. Ayrıca bu parametrelerle başlangıcı 2 olan bir anahtarın deneme dizisi $i = 0, \ldots, 6$ için 2, 5, 3, 3, 5, 2, 1'dir (python ile hesaplandı); yedi slotun yalnız dördü denenir. Hocanın "$c_1$, $c_2$ ve $m$ tablonun tamamı kullanılacak şekilde seçilmeli" uyarısının somut örneği budur.

**Ek bilgi (derste anlatılmadı):** Karesel denemede iki anahtarın başlangıç konumu aynıysa bütün deneme dizileri de aynıdır; bu, doğrusal denemedeki birincil kümelenmeden (primary clustering) daha hafif olan ikincil kümelenme (secondary clustering) diye adlandırılır. Çift hashing, ikinci fonksiyon anahtara göre değiştiği için bunu da büyük ölçüde ortadan kaldırır.

### 9.3 Çift Hashing (Double Hashing)

Çift hashing'de iki hash fonksiyonu, $h_1$ ve $h_2$, birlikte tek bir deneme fonksiyonu üretir. Diğer yöntemlere göre iyi sonuç verir, çünkü bu yöntemin oluşturduğu permütasyonlar, yani deneme seçenekleri, rastgeleye yakındır.

Örnekteki fonksiyonlar:

$$h_1(k) = k \bmod m, \qquad h_2(k) = 1 + (k \bmod m')$$

Birincisinde $m$'nin tablo boyutu olması önemlidir. İkincisinde yeni bir $m'$ devreye girer: $m$'den küçük, ama çok da küçük olmayan (slightly less) bir değer, örneğin $m-1$ gibi. Deneme $i$ için konum, hocanın hesaplarından görüldüğü gibi $h_1(k) + i \cdot h_2(k)$'nin $m$'ye göre modudur.

**Örnek** (ekrandan, 02:30:00 ve 02:35:40): Tablo boyutu 13, yani $m = 13$. $m-1 = 12$ seçilirse çift sayı elde edilir; asal kullanmak daha mantıklı olduğu için $m' = 11$ seçilir. Tabloda bazı yerler önceden doludur: slot 1'de 79, slot 4'te 69, slot 5'te 98, slot 7'de 72, slot 11'de 50. 14 eklenmek isteniyor.

- $h_1(14) = 14 \bmod 13 = 1$
- $h_2(14) = 1 + (14 \bmod 11) = 1 + 3 = 4$
- $i = 0$: $h(14, 0) = 1$. Slot 1 dolu (79).
- $i = 1$: $h(14, 1) = 1 + 1 \cdot 4 = 5$. Slot 5 dolu (98).
- $i = 2$: $h(14, 2) = 1 + 2 \cdot 4 = 9$. Slot 9 boş; 14 buraya yerleşir.

Çift hashing rastgeleliğe daha yakın sonuçlar ürettiği için tercih edilir. Hoca bir sonraki derste teoremler ve ispatlar üzerine devam edileceğini söyledi.

## Örnek Sorular

**Soru 1** (ekrandan, 00:26:40). $T(n) = 2T(\lfloor n/2 \rfloor) + n$, $T(1) = 1$ bağıntısının $O(n \log n)$ olduğunu tümevarımla yerine koyma yöntemiyle gösterin; uygun $c$ ve $n_0$'ı bulun.

> **Çözüm (derste verildi):** Varsayım $T(n/2) \le c\frac{n}{2} \log \frac{n}{2}$. $T(n) \le cn \log \frac{n}{2} + n = cn \log n - n(c-1) \le cn \log n$ ($c \ge 1$ için). $n_0 = 1$ olmaz ($T(1) = 1 > 0$). $T(2) = 4$, $T(3) = 5$. $c = 1$: $4 \le 2$ sağlanmaz; $c = 2$: $4 \le 4$ sağlanır. Sonuç: $c = 2$, $n_0 = 2$.

**Soru 2** (ekrandan, 00:31:30). Aynı bağıntı için $T(n) = O(n)$ tahmin edilirse ne olur?

> **Çözüm (derste verildi):** $T(n) \le 2\frac{cn}{2} + n = cn + n$. Bu, $n > 0$ için hiçbir zaman $cn$'den küçük ya da eşit olamaz; tahmin yanlıştır.

**Soru 3** (ekrandan, 00:40:00 ve 00:42:40). $T(n) = T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n)$ ($n \ge 120$), $T(n) = O(1)$ ($n < 120$) bağıntısının $O(n)$ olduğunu gösterin.

> **Çözüm (derste verildi):** Tahmin $T(n) \le cn$, doğrusal terim $an$. $T(n) \le cn/5 + 7cn/10 + 6c + an = cn - (cn/10 - 6c - an)$. Gereken koşul $cn/10 - 6c - an \ge 0$, yani $c \ge 60c/n + 10a$. $n \ge 120$, $c \ge 20a$ için $20a \ge 60 \cdot 20a/120 + 10a = 20a$ sağlanır.

**Soru 4** (derste sözlü, 01:09:03). $m$ uzunluğunda bir doğrudan adres tablosunda tutulan bir dinamik kümede en büyük elemanı bulmanın en kötü durumu nedir?

> **Çözüm (derste verildi):** Bütün tablo taranmak zorundadır; $O(m)$.

**Soru 5** (derste sözlü, 01:13:57 ve 01:16:17). Zincirlemede neden çift yönlü bağlı liste kullanılır? Tek yönlü liste kullanılsaydı silmenin karmaşıklığı ne olurdu?

**Çözüm (derste verildi):** Çift yönlü listede silinecek düğüm verildiğinde önceki ve sonraki düğüm elde vardır; iki bağlantı güncellenir, $O(1)$. Tek yönlü listede önceki düğüm tutulmadığı için liste baştan taranmalıdır; bütün anahtarlar aynı slottaysa $O(n)$.

**Soru 6** (ekrandan, 01:33:40 ve 01:40:00). Zincirlemeli bir hash tablosunda, basit düzgün hashing varsayımıyla başarılı bir aramanın ortalama $\Theta(1+\alpha)$ sürdüğünü gösterin.

**Çözüm (derste verildi):** $X_{ij}$ göstergesiyle ortalama inceleme sayısı $\frac{1}{n}\sum_{i=1}^{n}(1+\sum_{j=i+1}^{n}X_{ij})$. $E[X_{ij}]=1/m$ konunca $1+\frac{1}{nm}\sum_{i=1}^{n}(n-i)=1+\frac{n-1}{2m}=1+\frac{\alpha}{2}-\frac{\alpha}{2n}=\Theta(1+\alpha)$.

**Soru 7** (ekrandan, 02:00:30). $m=10000$, $A=(\sqrt{5}-1)/2$ ile çarpma yönteminde $k=123456$ anahtarının hash değeri nedir?

**Çözüm (derste verildi):** $123456\cdot 0{,}618033\approx 76300{,}0041151$; ondalık kısım $0{,}0041151$; $10000\cdot 0{,}0041151=41{,}151$; $h(k)=41$.

**Soru 8** (ekrandan, 02:17:55). $m=7$, $h'(k)=k \bmod 7$ ile doğrusal denemede 9, 16 ve 23 sırayla eklenirse hangi slotlara yerleşir?

**Çözüm (derste verildi):** 9 → 2; 16 → 2 dolu, $(2+1) \bmod 7=3$; 23 → 2 ve 3 dolu, 4. Arka arkaya dolu slotlardan bir küme oluşur.

**Soru 9** (ekrandan, 02:24:25). $c_1=2$, $c_2=1$, $m=7$ ile karesel denemede aynı anahtarlar (9, 16, 23) sırayla eklenirse hangi slotlara yerleşir?

**Çözüm (derste verildi, bir adım düzeltildi):** 9 → 2; 16 → $h(16,1)=2+2+1=5$; 23 → 2 ve 5 dolu, $h(23,2)=2+4+4=10$. Hoca mod almadı; $10 \bmod 7=3$, 23 slot 3'e yerleşir.

**Soru 10** (ekrandan, 02:35:40). $m=13$, $h_1(k)=k \bmod 13$, $h_2(k)=1+(k \bmod 11)$ ile çift hashing yapılıyor. Slot 1, 4, 5, 7 ve 11 dolu (79, 69, 98, 72, 50). 14 nereye yerleşir?

**Çözüm (derste verildi):** $h_1(14)=1$, $h_2(14)=4$. Denemeler: 1 (dolu), $1+4=5$ (dolu), $1+2\cdot 4=9$ (boş). 14 slot 9'a yerleşir.

**Soru 11** (derste sözlü, 02:07:47). Açık adreslemede silme nasıl yapılmalıdır?

**Çözüm (derste verildi):** NIL konursa arama erken biter. Yerine DELETED işareti konur; insert bu slotu boş sayar. Silme çoksa arama süresi silinen elemanlara bağlı hâle gelir; bu durumda zincirleme tercih edilir.

## Dersin Toparlaması

Dersin ilk bölümünde, master teoreminin uygulanamadığı bağıntılar için tümevarımla yerine koyma yöntemi öğrenildi: önce çözümün biçimi tahmin edilir, tahmin $n$'den küçük değerler için doğru kabul edilip $n$ için gösterilir, sonra başlangıç koşullarını sağlayan $c$ ve $n_0$ bulunur. Merge sort örneğinde $c=2$, $n_0=2$ ile $O(n\log n)$; yanlış bir $O(n)$ tahmininde ise ispatın nasıl tıkandığı görüldü. Geçen dersten kalan medyan bulma bağıntısı da bu yöntemle çözüldü: $n\geq 120$ ve $c\geq 20a$ için $T(n)=O(n)$.

İkinci bölümde bağlı listeler hatırlatılarak hash tablolarına geçildi. Doğrudan adreslemenin bellek sorunu, hash fonksiyonu ve çakışma kavramlarına götürdü. Çakışmalar zincirlemeyle (çift yönlü bağlı listeler, başa ekleme) ya da açık adreslemeyle (doğrusal, karesel, çift hashing) çözülür. Basit düzgün hashing varsayımı altında zincirlemede hem başarısız hem başarılı aramanın ortalama $\Theta(1+\alpha)$ olduğu ispatlandı. Hash fonksiyonu olarak bölme ve çarpma yöntemleri incelendi. Sonraki derste universal hashing ile hash tablolarına ilişkin teoremler ve ispatlarla devam edilecek.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.